



¿QUÉ ES GIT?

Git es un sistema de control de versiones distribuido creado por Linux Torvalds en 2005 (el mismo creador de Linux). Su función principal es gestionar los cambios en el código fuente de un proyecto a lo largo del tiempo.

A diferencia de guardar archivos manualmente en carpetas como “proyecto-final”, “proyecto-final2”, “proyecto-final-último”, Git permite llevar un historial completo y ordenado de versiones del proyecto, con la posibilidad de volver atrás, comparar cambios o trabajar en paralelo sin perder información.

Importancia de Git en el desarrollo de software en equipo

1. Historial seguro de cambios Cada modificación en el proyecto queda registrada con detalles: quién la hizo, cuándo y por qué. Esto evita perder trabajo y facilita rastrear errores.
2. Trabajo colaborativo sin conflictos Varios desarrolladores pueden trabajar al mismo tiempo en diferentes partes del código, gracias a las ramas (branches). Al final, esas ramas se pueden unir (merge) de forma ordenada.
3. Prevención de errores Si un cambio introduce un error, se puede volver a una versión anterior del proyecto fácilmente (rollback), sin afectar a todo el equipo.
4. Integración con plataformas colaborativas Git se integra con servicios como GitHub, GitLab o Bitbucket, que permiten revisar código, reportar problemas (issues), automatizar pruebas y desplegar proyectos.
5. Escalabilidad Desde proyectos personales hasta sistemas de millones de líneas de código, Git permite mantener el orden, la calidad y la trazabilidad en el desarrollo

Ejemplo práctico en un equipo de desarrollo

Imagina que un equipo de 5 programadores está construyendo una aplicación:

“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



- Ana crea la interfaz de usuario.
- Pedro desarrolla la lógica de autenticación.
- María trabaja en la base de datos.
- José escribe pruebas automatizadas.
- Laura se encarga de la documentación y estilos.

Con Git:

- Cada uno trabaja en su rama independiente.
- Pueden hacer cambios sin interferir con el trabajo de los demás.
- Cuando sus cambios están listos, se integran en la rama principal (main o master) mediante revisiones de código.
- Si algo falla, siempre se puede regresar a una versión estable del proyecto

Conclusión

Git es la columna vertebral del desarrollo de software moderno, porque permite a los equipos colaborar de forma organizada, segura y eficiente, asegurando la calidad del código y la trazabilidad de cada cambio.

¡¡¡ Empecemos en viaje a este nuevo mundo !!!

Recuerda:

*“Cada línea que aprendes hoy es una herramienta más para construir el futuro que sueñas.
Estudiar no es una obligación, es la llave que abre las puertas de tus oportunidades.”*

“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



INSTALACIÓN DE GIT

1. Dirigirse al link: <https://git-scm.com/downloads>
2. Descargar la versión compatible con su sistema operativo:

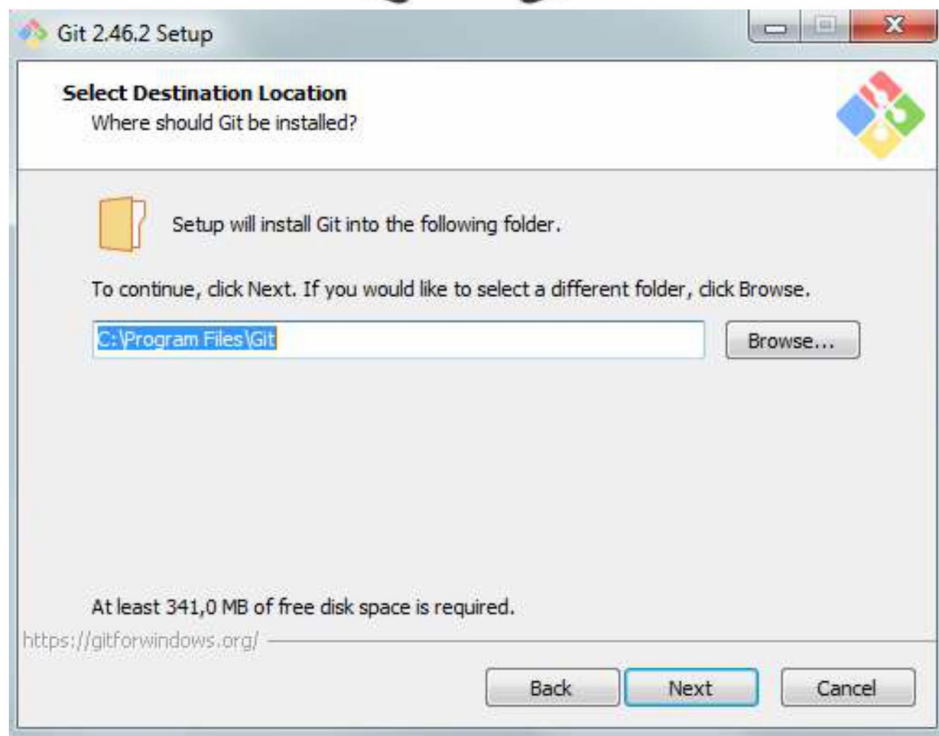


3. Da doble clic al archivo ejecutable que se descarga
4. En las siguientes ventana hacer clic en next

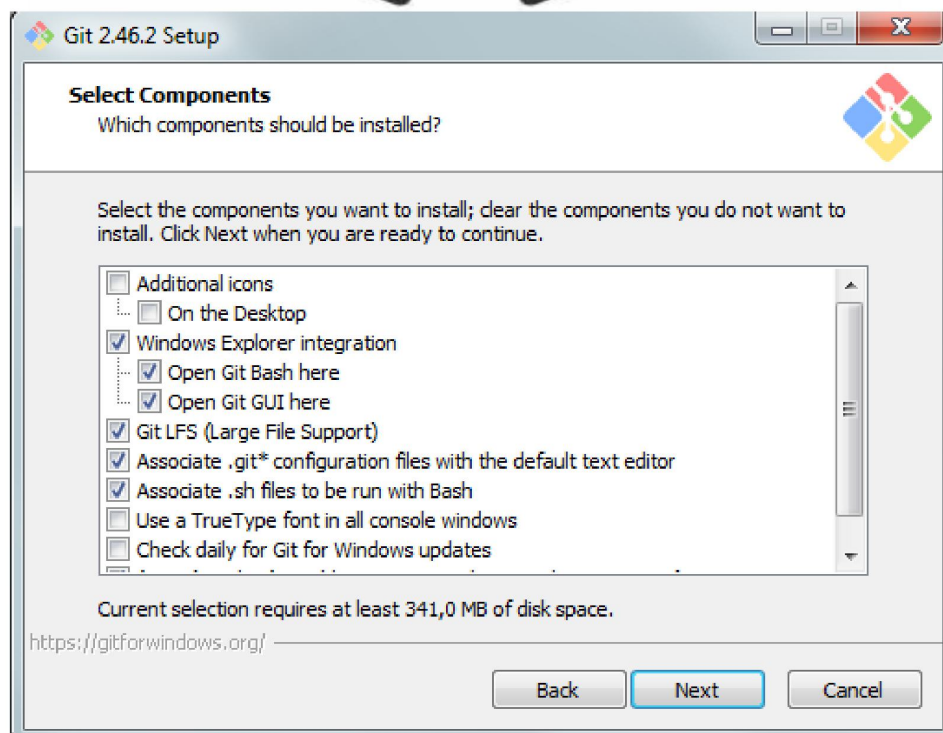
“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



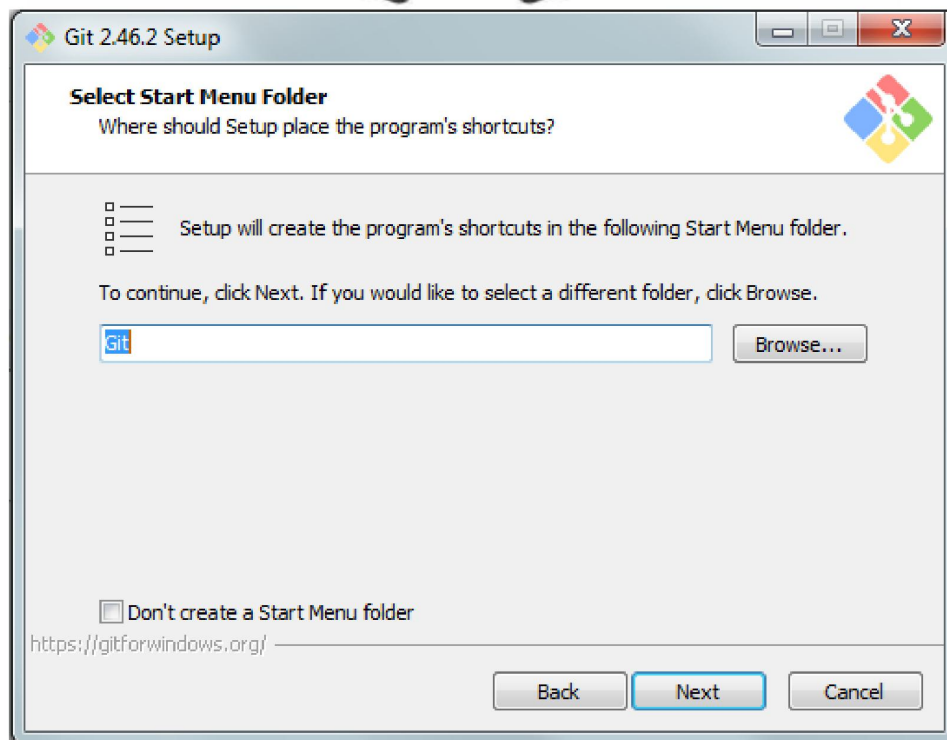
“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



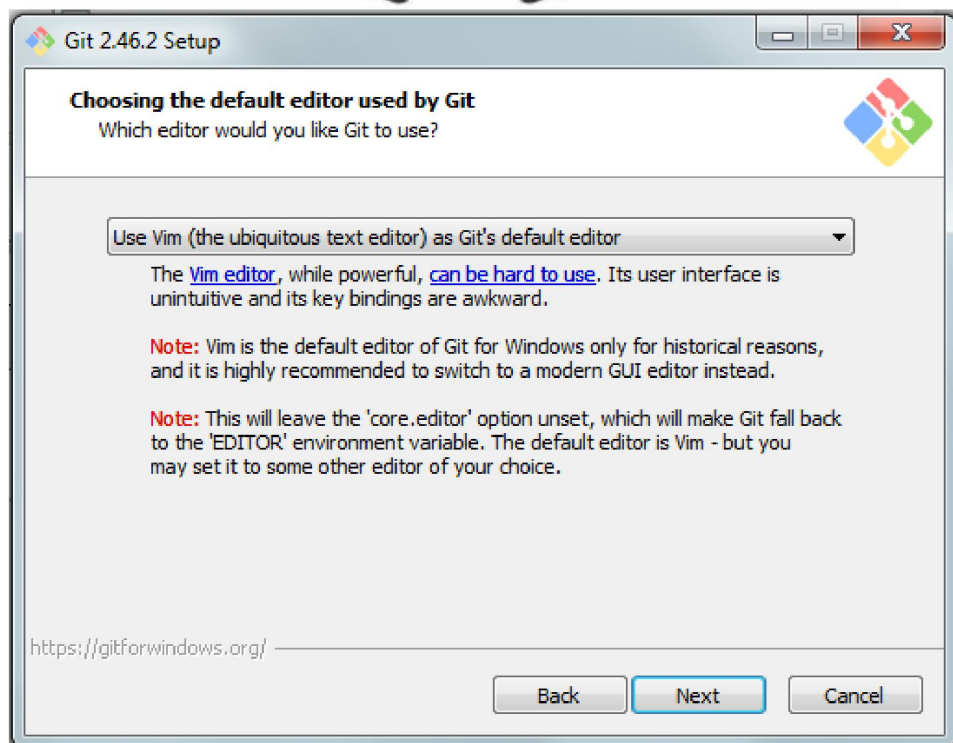
“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



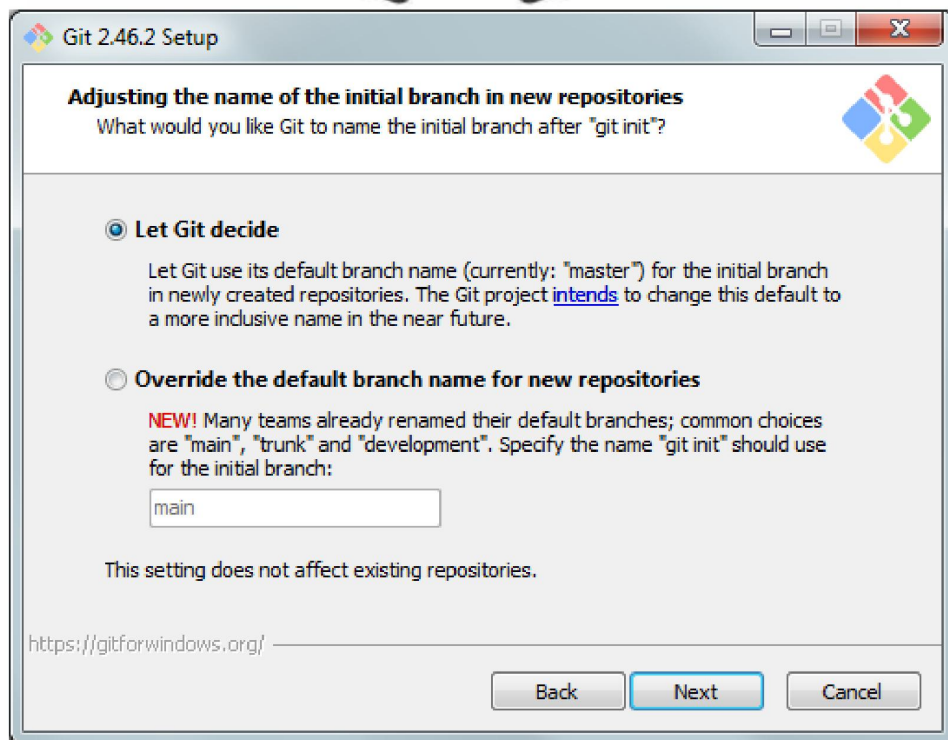
“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



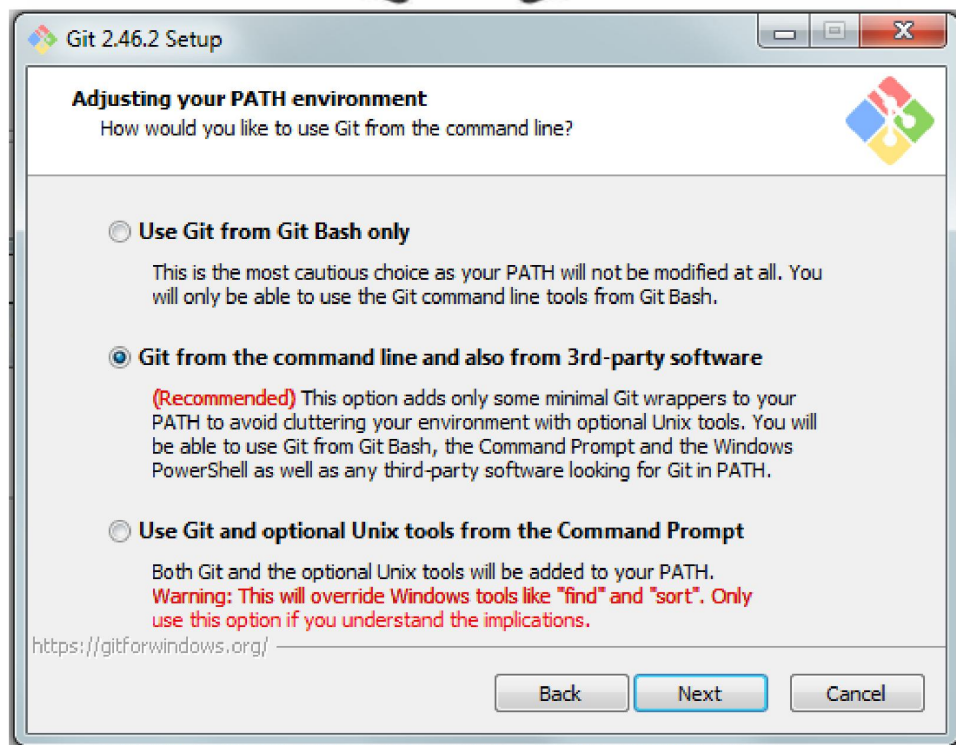
“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



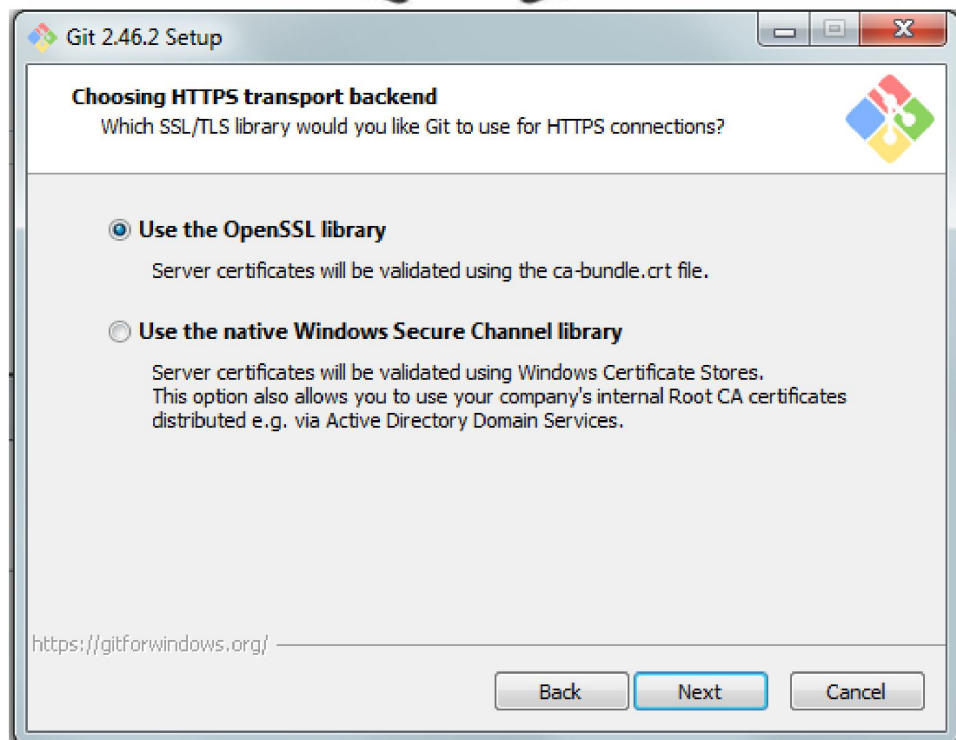
“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



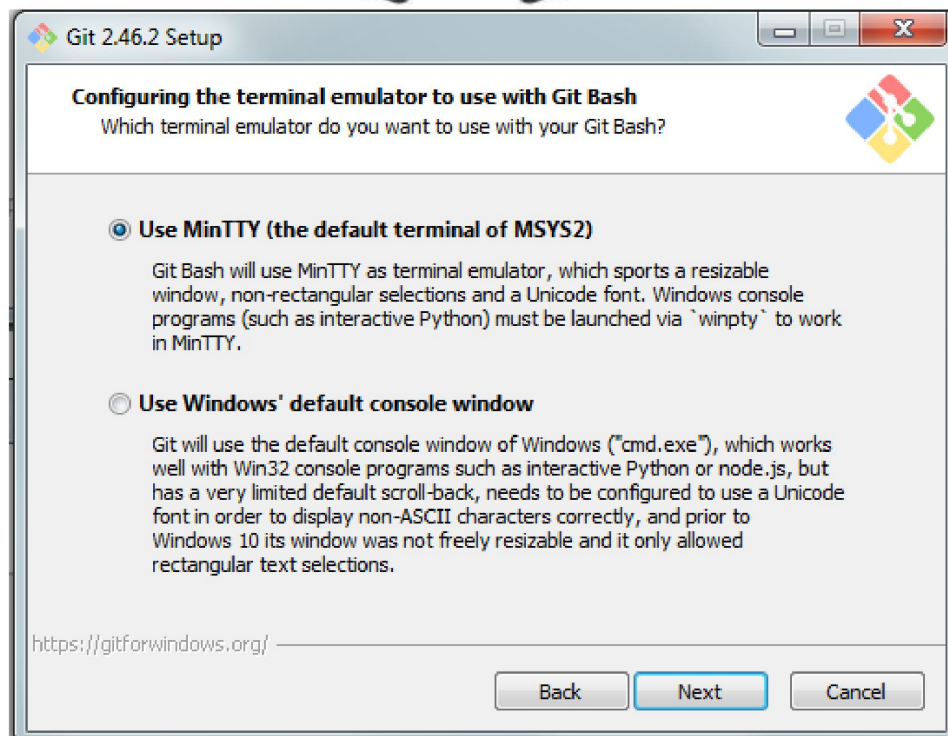
“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



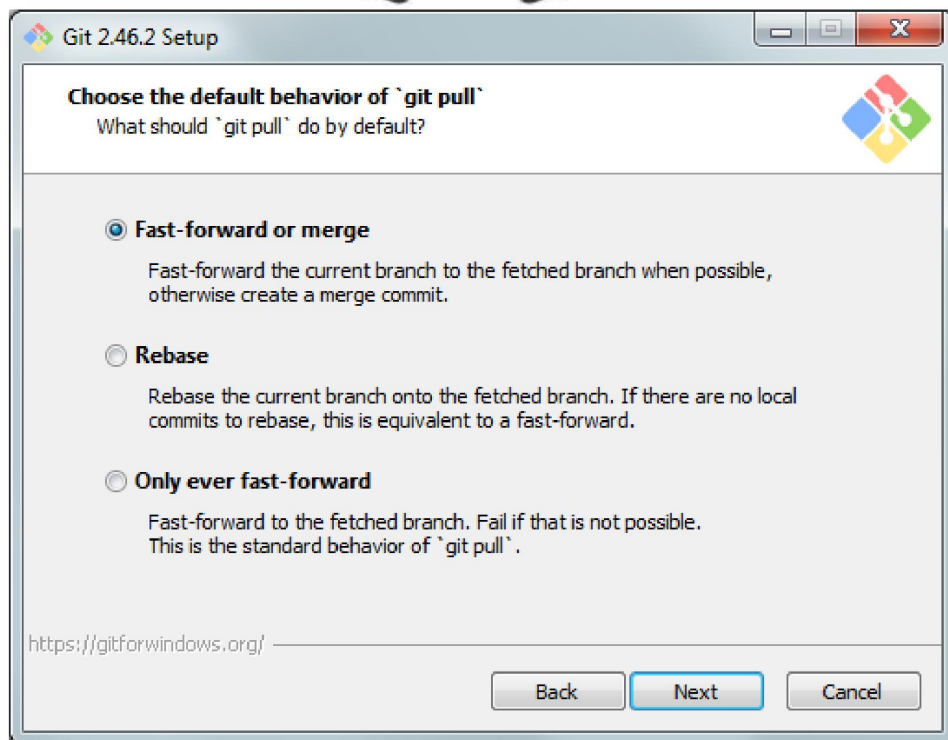
“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



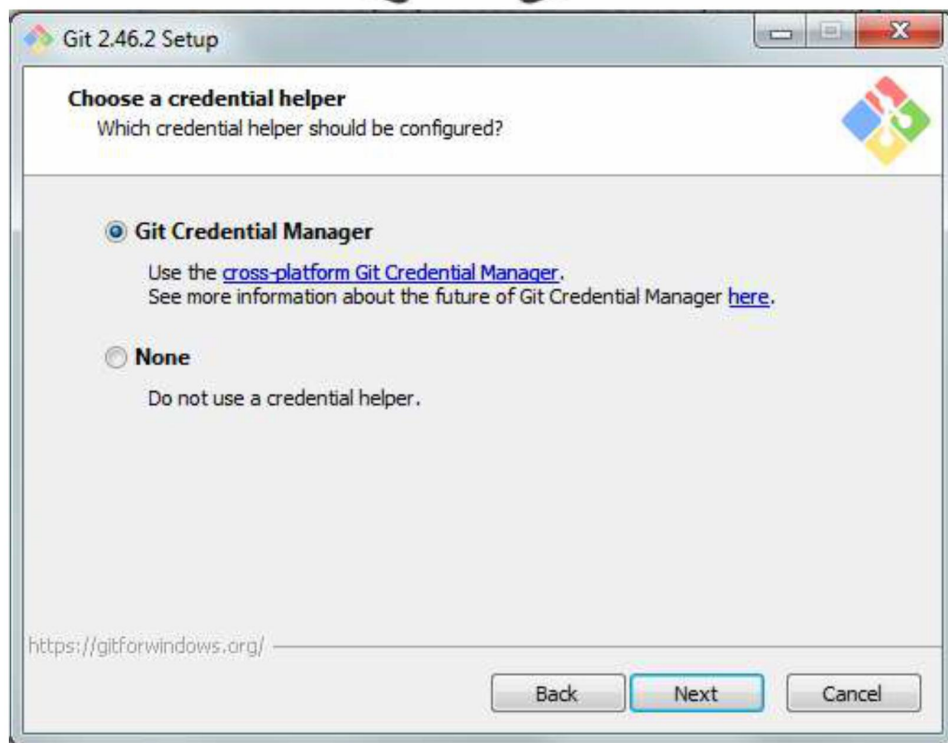
“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



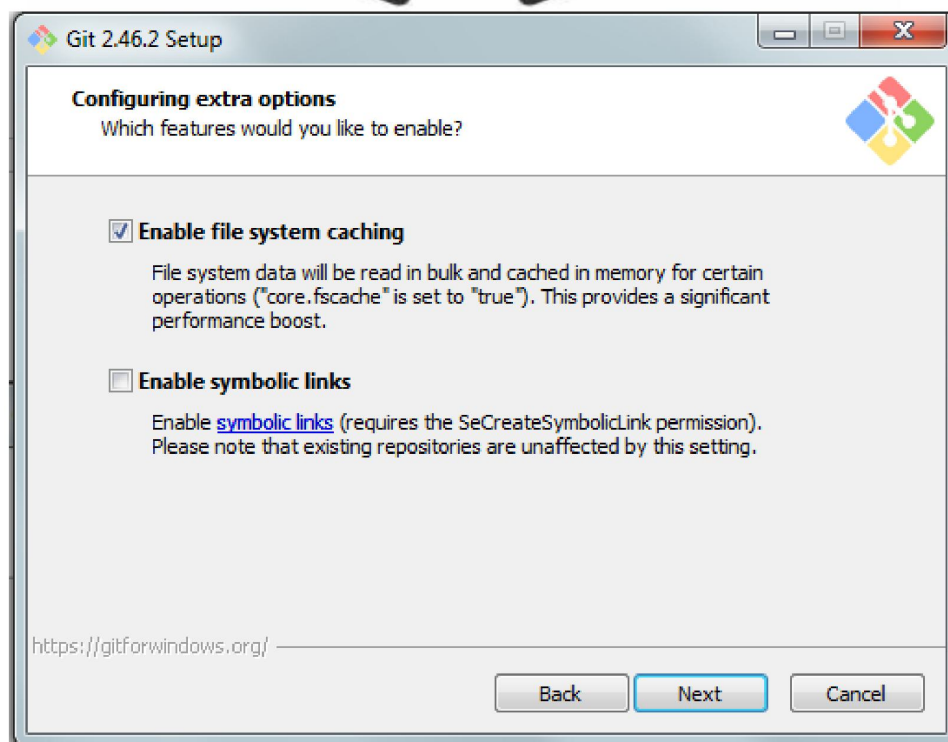
“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”

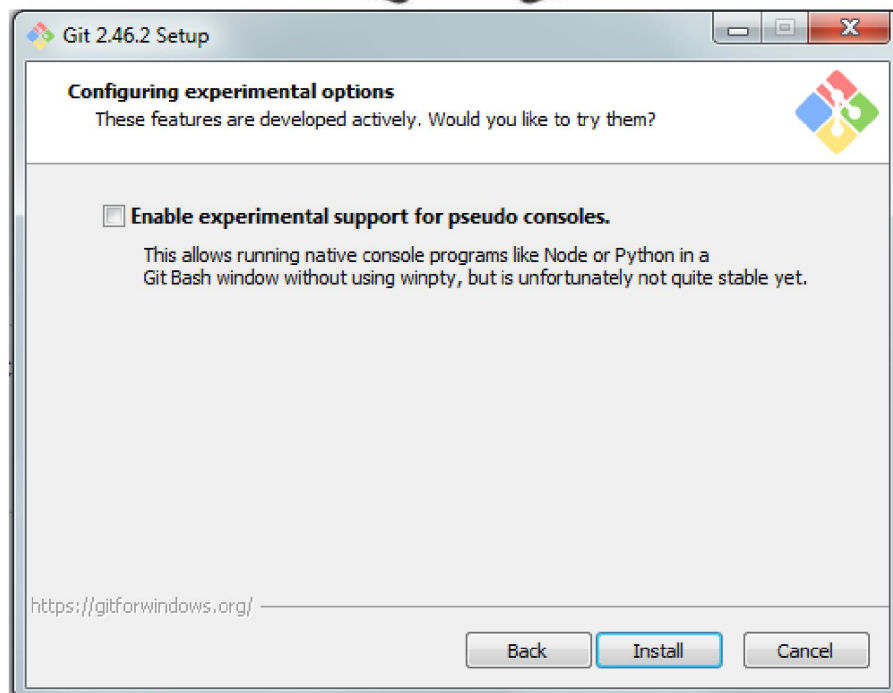


“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



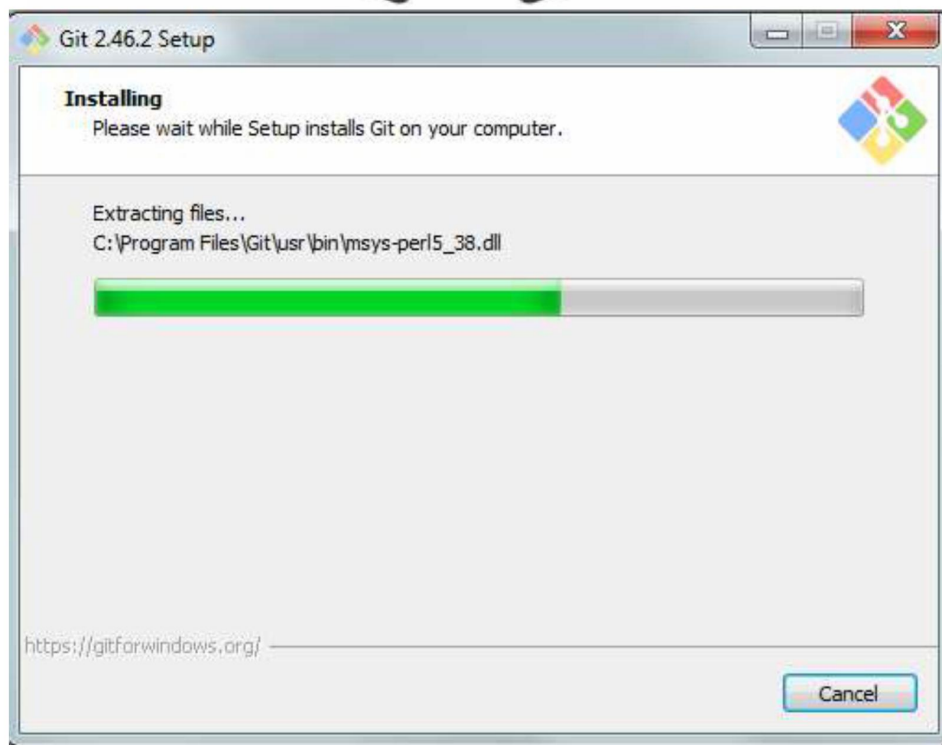
En la siguiente ventana hacer clic en install

“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



Esperar que realice el proceso:

“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



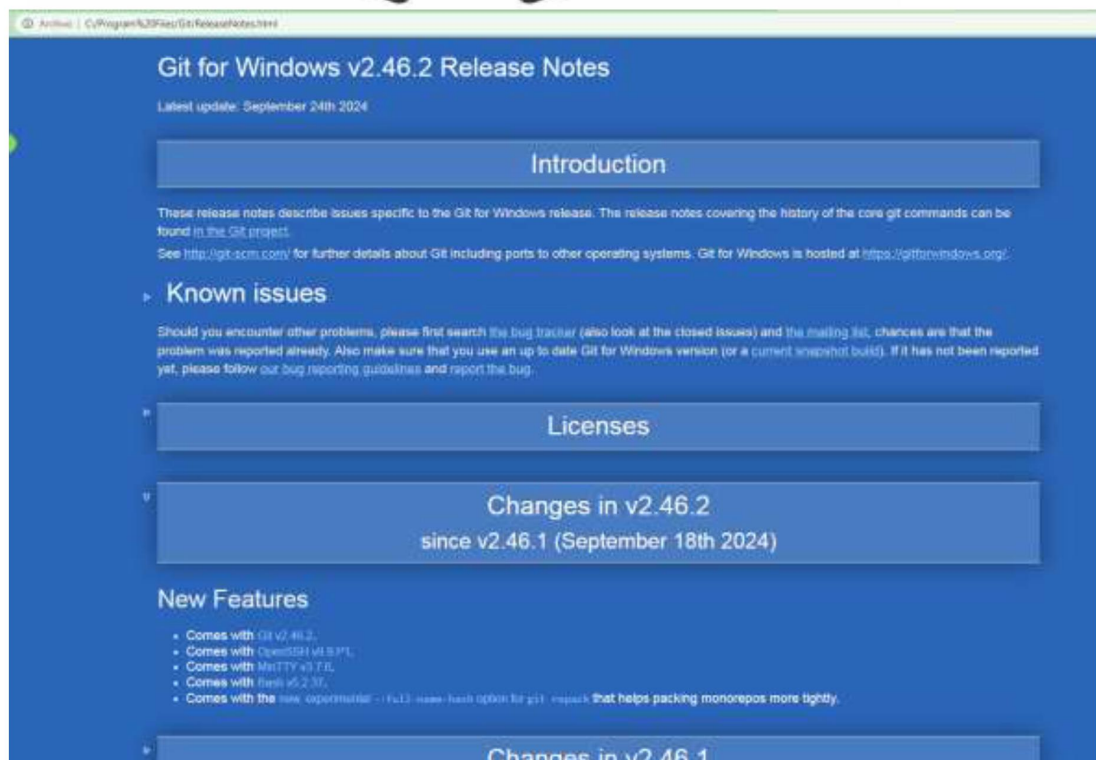
Hacer clic en finish

“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



Te abre una pestaña en el navegador de información sobre la versión, la puedes cerrar.

“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



Listo ya puedes utilizar git!!!!

Comandos básicos de Git

- **Configuración inicial (solo la primera vez):**

Identifica al autor de los cambios. El --global aplica a todos los proyectos en tu PC

`git config --global user.name "Tu Nombre"`

“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



`git config --global user.email tuemail@ejemplo.com`

Debes reemplazar por tu nombre y tu correo electrónico

- **Crear o clonar un repositorio:**

El siguiente comando Inicializa un repositorio Git en la carpeta actual.

`git init`

El siguiente comando descarga un repositorio existente de GitHub/GitLab/Bitbucket.

`git clone https://github.com/usuario/repositorio.git`

- **Flujo básico de trabajo:**

El siguiente comando muestra el estado de los archivos (nuevos, modificados, preparados o sin cambios):

`git status`

El siguiente comando prepara un archivo específico para el próximo commit:

`git add archivo.txt`

El siguiente comando prepara todos los archivos modificados/nuevos:

`git add .`

El siguiente comando registra los cambios en el historial con un mensaje:

`git commit -m "Mensaje descriptivo"`

- **Ver historial y cambios**

“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



Mostrar el historial de commits:

git log

Mostrar las diferencias entre lo modificado y lo registrado:

git diff

- **Ramas (branches):**

Listar las ramas existentes:

git branch

Crear una nueva rama:

git branch nueva-rama

Cambiar a la rama indicada:

git checkout nueva-rama

Crea y cambia a una nueva rama directamente:

git checkout -b nueva-rama

Fusiona los cambios de una rama en la actual:

git merge nombre-rama

- **Conexión con repositorios remotos**

Comando que muestra las direcciones del repositorio remoto:

git remote -v

“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



Comando que sube los cambios locales a la rama indicada en el remoto:

`git push -u origin main`

Descarga e integra los cambios del remoto a tu rama local

`git pull origin main`

Descarga los cambios del remoto sin fusionarlos aún.

`git fetch`

- **Comandos útiles extra**

Saca un archivo del área de preparación:

`git reset archivo.txt`

Explicación:

Si el archivo estaba en el área de preparación (staging área, con git add), Lo saca del área de preparación, pero no borra el archivo ni sus cambios. Es decir, el archivo vuelve al estado de “modificado pero no preparado”.

Ejemplo:

`git add archivo.txt`

`git status`

ahora archivo.txt aparece en "Changes to be committed"

“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



```
git reset archivo.txt
```

```
git status
```

```
# ahora archivo.txt aparece en "Changes not staged for commit"
```

Comando que elimina un archivo del proyecto y lo marca para commit:

```
git rm archivo.txt
```

Comando que guarda temporalmente cambios sin confirmar para trabajar en otra cosa:

```
git stash
```

Nota: Con estos comandos básicos, ya puedes trabajar con Git en cualquier proyecto.

- Tener en cuenta:

El flujo más usado en un equipo es:

```
git pull: traer lo más reciente
```

```
git add : preparar cambios
```

```
git commit -m "mensaje" : registrar cambios
```

```
git push : enviar al repositorio remoto
```

“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



GITHUB

GitHub es una plataforma en la nube que permite almacenar, gestionar y colaborar en proyectos de software que usan Git como sistema de control de versiones.

Fue creada en 2008 y actualmente pertenece a Microsoft. En pocas palabras: Git es la herramienta que gestiona las versiones del código, y GitHub es el servicio que facilita el trabajo colaborativo en línea.

Importancia de GitHub en el desarrollo de software

1. Colaboración en equipo

Permite que múltiples desarrolladores trabajen en el mismo proyecto desde cualquier lugar del mundo.

Usa ramas (branches) y solicitudes de cambios (pull requests) para integrar el trabajo de forma ordenada.

2. Almacenamiento seguro en la nube

Tu código no depende solo de tu computador.

Está respaldado en servidores y accesible en cualquier momento.

3. Historial y trazabilidad

Registra quién hizo cada cambio, cuándo y por qué.

Facilita encontrar el origen de errores y comprender la evolución del proyecto.

4. Revisión de código y calidad

“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



Con las pull requests, los equipos revisan y comentan el código antes de integrarlo.

Se fomenta la mejora continua y la calidad del software.

5. Integración con herramientas de desarrollo (DevOps)

GitHub se conecta con sistemas de integración continua (CI/CD), pruebas automáticas, despliegue en servidores, documentación y tableros de tareas.

Ejemplo: GitHub Actions permite automatizar procesos como pruebas o despliegues.

6. Comunidad y aprendizaje

GitHub es también una red social para programadores.

Puedes explorar proyectos de código abierto, aprender de otros desarrolladores y contribuir a comunidades.

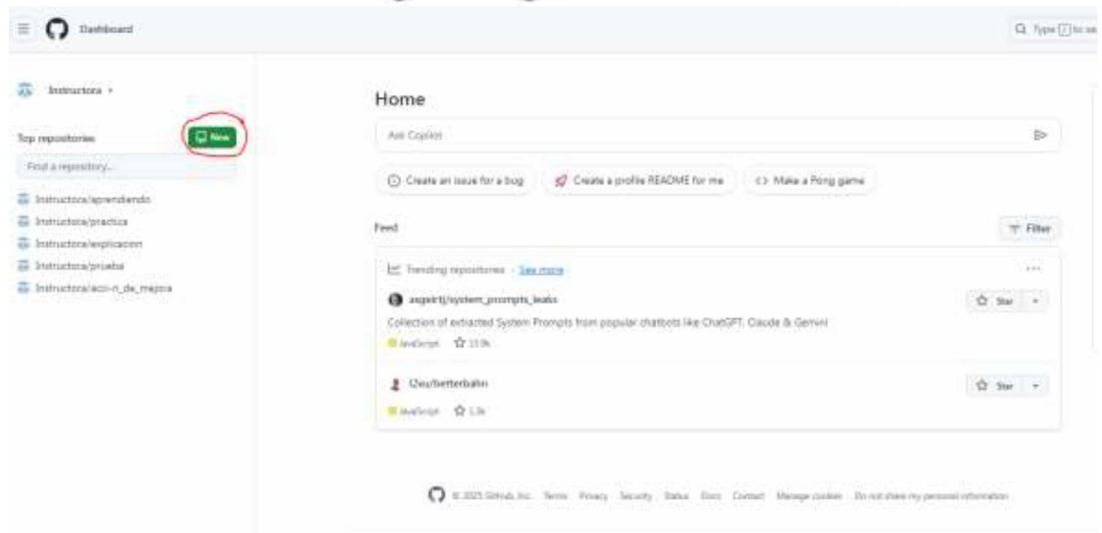
Conclusión:

GitHub no es solo un sitio para guardar código, es la plataforma central de colaboración, control de calidad y automatización en la mayoría de equipos de desarrollo modernos.

Creación de un repositorio en github

1. Crear una cuenta en github <https://github.com/>
2. Iniciar sesión
3. En el botón señalado, haz clic:

“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



4. Aparece la siguiente información:

“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

1 General

owner and name **Instructora** /

Great repository names are short and memorable. How about **silver-funicular**?

Description

0 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

Add README ☐ Off

READMEs can be used as longer descriptions. [About READMEs](#)

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

Add license

Licenses explain how others can use your code. [About licenses](#)

[Create repository](#)

Deberás colocar el nombre del repositorio, realizar una descripción si lo deseas, dejarlo público. Después de realizar lo anterior, debes dar clic en el botón **Create Repository**

Y nos debe aparecer la siguiente información:

“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”



Listo !!!!

Tienes el material de apoyo necesario para empezar a trabajar con git

“Material exclusivo de apoyo para la formación, elaborado por la Instructora Isaura Suarez. Prohibida su reproducción total o parcial sin autorización.”